Ders 6 Genişletilmiş Notu: Davranış Modellemesi, Durum Diyagramları ve Mimari Stillere Giriş

Ders 6 Genişletilmiş Notu: Davranış Modellemesi, Durum Diyagramları ve Mimari Stillere Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin planı
- 2. Davranış modellemesi neden gerekir?
- 3. Durum kavramı ve durum patlaması
- 4. Olay kavramı
- 5. Durum mu olay mı?
- 6. Reaktif sistemler ve üç modelleme seviyesi
- 7. Kombinatoryal sistemler: karar tablosu ve karar ağacı
- 8. Ardışık sistemler: garaj kapısı
- 9. Durum geçiş diyagramı ve telefon örneği
- 10. Durum diyagramları (statecharts)
- 11. İç içe gruplama
- 12. Eş zamanlılık
- 13. Senkronizasyon: yayım (ardarda) olaylar
- 14. Koşullu ve özel geçişler
- 15. Geçmiş (history)
- 16. UML'de durum ve geçiş tanımı
- 17. Sınıf diyagramı ile ilişki
- 18. Casio F-91W dijital saat örneği
- 19. Ara özet: reaktif sistemleri modellemek
- 20. Radyo saat örneği
- 21. Geçerleme ve davranış modellemesinin adımları
- 22. Yazılım mimarisi nedir?
- 23. Mimarinin resmî tanımları
- 24. Bileşenler, API'ler ve bağlayıcılar
- 25. Kavramsal mimari, sapma ve erozyon
- 26. Mimari görünümler
- 27. Mimari stiller
- 28. Mimari stil kataloğu
- 29. Heterojen sistemler, ADL'ler ve mimari değerlendirme
- Örnek Sorular

Ders üç bölümden oluşur. İlk bölümde BLM5126-07 slayt destesiyle (69 slayt) davranış modellemesi ve durum diyagramları anlatılır (ekrandan, 00:10:00). Aradan sonra aynı destenin sonundaki radyo saat örneği üzerinden bir durum diyagramının adım adım nasıl kurulduğu gösterilir. Son bölümde BLM5126-08 destesine (47 slayt) geçilir ve yazılım mimarisinin tanımı ile mimari stiller tanıtılır (ekrandan, 02:00:00).

1. Dersin planı

Hoca günün planını şöyle verir: önce davranış modellemesi, durum diyagramlarının ne olduğu, nasıl oluşturulduğu ve model kurgulanırken neler kattığı; sonra durum diyagramıyla ilgili küçük bir örnek; ardından klasik anlamda yazılım mimarisinin tanımı ve mimari stillerin isimleri ve ana hatları.

Sonraki konular olarak mimari görünümler (çoğu UML ile zaten görüldü), mimariyi ifade etmek için kullanılan dillerden biri olan Acme'nin kısa tanıtımı ve orta katman çözümleri anılır. Hoca, mimari stillerin genel yaklaşımlarından sonra günümüzde yeni teknolojilerle standart olarak kullanılan mikroservis temelli mimarilerin ve burada uygulanması gereken tasarım yaklaşımlarının daha ayrıntılı inceleneceğini söyler.

2. Davranış modellemesi neden gerekir?

Şimdiye kadar bir problem anlaşılmaya çalışıldı, sonra çözüm yolu bir tasarım olarak ortaya konup modellendi. Hoca bunların bir önceki dersteki OCL değişmezleri gibi düşünülebileceğini söyler: sistemin sabit, değişmeyen özelliklerini tanımlarlar.

Oysa genellikle tepkisel (reaktif) sistemler geliştirilir: dış uyarılara tepki veren, bunlara karşı davranış sergileyen sistemler. Özetle sorulması gereken soru şudur: **bir sistem dış uyarılara nasıl tepki vermelidir?** Bunu bir diyagramla modellemek ve kesin tanımlar yapmak gerekir; bunun için kullanılan diyagram durum diyagramıdır. Bunun için önce durum ve olay kavramlarından başlamak gerekir.

3. Durum kavramı ve durum patlaması

Slayttaki tanım şudur (ekrandan, 00:10:00): **Durum, verilen bir anda sistem değerleri kümesinin soyut bir tanımıdır.** Hoca bunu "herhangi bir T anında sistemdeki bütün değerlerin bir araya gelmesiyle oluşan vaziyet" diye açar ve kendisinin de tanımı yaparken neredeyse döngüselliğe (circularity) düştüğünü, kavramı kendi adıyla tanımlamaya çalıştığını söyler. Özetle durum, herhangi bir andaki sistemin hâlidir; bütün değişkenlerin aldığı değerlerin bir karşılığı olarak düşünülebilir.

Slayttaki örnek "Dışarıda yağmur yağıyor" cümlesidir. Bu, zamanda seçilen bir noktada (ör. saat 10.09'da), seçilen bir konumda havadaki yağış miktarı için bir tahmin ya da soyutlamadır. Havanın durumu söylenmektedir ve bu basit örnekte iki durum vardır: yağmurlu ya da değil.

3.1 Kasabadaki arabalar

Daha karmaşık bir örnek: New York'taki Manhattan gibi birbirini dik kesen sokaklardan oluşan hayali bir kasaba olsun; 10 sokak ve 8 cadde vardır. İçinde tek bir araba dolaşmaktadır ve herhangi bir anda örneğin 3. sokak ile 5. caddenin kesiştiği noktadadır.

- Verilen: 10 sokak, 8 cadde, 1 araba.
- İşlem: arabanın bulunabileceği kesişim sayısı 10×8.
- Sonuç: 80 olası durum. Hava örneğinde iki durum vardı; tek bir değişkenle durum sayısı 80'e çıktı.

İki araba olursa her birinin 80 durumu olduğundan $80 \times 80 = 6400$ durum elde edilir. Hoca, iki araba aynı yerde bulunamayacağı için "6400 - 1, yani 6399" diyebileceğinizi, ama kabaca 6400 dediğini ekler.

> **Ders sonrası not:** İki arabanın aynı yerde olduğu hâllerin sayısı 1 değil 80'dir. Arabalar ayırt edilebilirse durum sayısı büyük olasılıkla $80 \times 79 = 6320$, ayırt edilemezse $\binom{80}{2} = 3160$ olur (python ile doğrulandı). Hocanın "kabaca 6400" demesi bu nedenle yerindedir.

Araba sayısı arttıkça durum sayısı katlanarak büyür. Bir sistemdeki değişken sayısı ve değişkenlerin alabileceği değerler arttıkça durum uzayı üstel olarak büyür. Buna **durum patlaması problemi (state explosion problem)** denir. Herhangi bir T anında sistemin bütün değişkenlerinin alacağı değerlerle birlikte ne hâlde olacağı tek tek modellenmeye çalışılırsa işin içinden çıkılamaz.

3.2 SOS oyunu

Bir örnek daha: 3x3'lük bir SOS tahtası, toplam 9 kare. Her gözde S, O ya da boş olabilir; yani her gözün 3 değeri vardır. Oyunun kuralları (ör. S'nin yanına O yazılırsa kaybetmek) düşünülmezse:

- Verilen: 9 göz, her göz için 3 değer.
- İşlem: 3^9.
- Sonuç: 19683 olası durum.

Her durumu tek tek yazmak büyük bir problem yaratır. Bunun yerine yalnız olabilecek durumlar üzerine bir modelleme yapılacaktır.

4. Olay kavramı

Olaylar durumların oluşmasına sebep olur. Tekil, ani, atomik bir oluşuma olay denir; fark edilebilir olmalı, bir şey yapan bir davranış olmalıdır. Sistem buna tepki verir. Olaylar sistemi uyaran dış uyaranlar olabileceği gibi iç uyaranlar da olabilir.

Olaylar iki çeşittir:

- **Asenkron olaylar:** herhangi bir anda, rastgele oluşan, bir anda yığılabilen ya da zamana yayılabilen olaylardır.
- **Senkron olaylar:** periyodik aralıklarla olan olaylardır. Genellikle bir sayaca ya da saate bağlıdır, zamanlanmıştır. Örnek: her saat başında bir alarm verilip buna göre bir şey yapılması.

Olayların temeli, sistemde bir durum değişikliği yaratmalarıdır. Buna **durum geçişi (state transition)** denir. Hocanın günlük hayattan örneği sınav sonucudur: sonuç sayfasına girip bakabildiğinizde bir anda "liseden mezun" durumundan "üniversite öğrencisi" ya da "kayıt yapma adayı" durumuna geçersiniz. Bu bir olaydır ve siz de bir sistem olarak durum değiştirirsiniz.

Olayları kim yaratır?

- Kullanıcılar: butona basmak, bir şeyi sürüklemek, bir şey yazmak.
- Verideki değişiklik: örneğin bir termostatta sıcaklık değeri 100 dereceyi geçti; bu sizin için bir olay yaratabilir.
- Zaman.

Olaylar zaman içinde biriktirilebilir; her birine oluştuğu anda tepki vermek gerekmeyebilir.

UML açısından dört olay tipi vardır:

- **Sinyaller:** dış etkenlerle oluşan bütün asenkron bildirimler.
- **Metot çağrıları:** senkron olarak bir işlemi yürütme ya da başlatma.
- **Veri durumları (data condition):** sıcaklığın 100 dereceyi geçmesi gibi.
- **Zaman akışı (timeline):** zaman içinde periyodik olarak gerçekleşecek olaylar. Hoca, timing diyagramından da daha önce bahsettiğini hatırlatır.

5. Durum mu olay mı?

Hoca slayttaki örnekleri tek tek tartışır (ekrandan, 00:20:00):

İfade	Cevap
Jesse Owens'ın Berlin'de 1936'da 100 metre olimpiyat şampiyonu olması	Durum
Mor renk	Hiçbiri; sadece bir özelliğin değeri
Binanın yangın söndürme sisteminin bir yangın nedeniyle devreye girmesi	Olay
Telefonun çalması	Olay
Yağmurlu bir gün	Durum
Saatin zilinin 12'yi vurması	Olay
Uluslararası Uzay İstasyonu	Hiçbiri; sadece çok karmaşık bir nesne

İfade	Cevap
Bilgisayarınızın ekran koruyucusu açılıyor	Slaytta cevap yok; hoca tetikleyici bir olay olarak açıklar

Açıklamalar:

- **Jesse Owens:** hoca Jesse Owens açısından bakar. Koşarken normal bir atletti; bitiş çizgisini geçtikten sonra olimpiyat şampiyonu durumuna geçti. Rekor kırması ise bir olaydı. Hoca, bu rekorun daha sonra kırıldığını ve 10 saniye sınırının altına inmek gibi düşünülebileceğini söyler.
- **Mor renk:** hiçbir şey değildir, yalnız bir özelliğin aldığı değerdir; yani bir durumun parçasıdır.
- **Yangın söndürme sistemi:** sensör yüzünden devreye girer; bir uyaran sayesinde olduğu için olaydır. Sistem açısından ise belli bir durumdaydı (bekleme) ve şimdi ıslatma ya da söndürme durumuna geçti. Tetiği düşünürsek olaydır ve sistemin durum değiştirmesine yol açmıştır.
- **Telefonun çalması:** olaydır; sonuçta ya açarsınız ya açmazsınız, yani ya konuşma durumunda olursunuz ya olmazsınız.
- **Saatin 12'yi vurması:** bir tetikleme mekanizması vardır, bu yüzden olaydır.
- **Uluslararası Uzay İstasyonu:** ne durum ne olaydır; yalnız çok karmaşık bir nesne ya da sistemdir.
- **Ekran koruyucusu:** bilgisayar bekleme durumuna geçer; açılma işlemi tetikleyici bir olay olarak karşımıza çıkar.

  Ders sonrası not: Hoca Owens'ın rekorunun "10 saniye sınırının altına inilmesiyle" kırıldığını söyledi. Owens'ın 1936 Berlin derecesi büyük olasılıkla 10,3 saniyedir ve 100 metrede 10 saniyenin altına ilk inen atlet büyük olasılıkla 1968'de Jim Hines'tır; yani Owens'ın kendisi 10 saniyenin altına inmemiştir.

6. Reaktif sistemler ve üç modelleme seviyesi

Sistemde durumlar ve durum geçişlerini sağlayan, tepki verilen olaylar vardır. Tanım olarak olaylara tepki veren sistemlere **reaktif sistemler** denir. Normalde işlerin sırasını kontrol etmenin sorumluluğu sistemin kendisindedir; reaktif sistemlerde ise uyaranlar dışarıdan gelir.

Reaktif ya da olay tabanlı sistemler modellenirken üç seviye vardır:

- **Kombinatoryal sistemler:** en basitidir. Yalnız girdiler ve onlara karşılık çıktılar vardır; sonraki durum yalnız girdilerle oluşur. Durumun geçmişi, yani önceki olaylar, sonraki durumu etkilemez.
- **Ardışık (sequential) sistemler:** bir bellek ya da tarihçe vardır; olaylar sıralı bir akış içinde birbirini izler.
- **Eş zamanlı (concurrent) sistemler:** asenkron ve eş zamanlıdır; pek çok olay ve durum geçişi vardır ve bunlar herhangi bir anda oluşabilir. Bunları önceden kestirmek mümkün değildir; periyodik bir yapı yoktur. Hoca, genel olarak tasarlanan sistemlerin bunlar olacağını söyler.

7. Kombinatoryal sistemler: karar tablosu ve karar ağacı

Kombinatoryal sistemler genellikle karar ağacı ya da karar tablosuyla modellenir; hoca ikisinin aslında aynı şey olduğunu söyler.

Slayttaki karar tablosu tanımı (ekrandan, 00:22:40): sistemin davranışını etkileyebilecek pek çok durum olduğunda kullanılır; girdi koşulları ve bu girdilere göre çıktı tepkileri hâlinde tanımlanır. Girdi kombinasyonları sonucu oluşturma şeklini belirler (kombinatoryal). Tablonun sütunlarında girdiler ve çıktılar, her satırında farklı bir girdi kombinasyonu yer alır.

Örnek: bir atölyede üç şalter vardır: ana şalter, ışık şalteri ve güç şalteri. Işık şalteri açılınca ışık yanar, güç şalteri açılınca motor çalışır. Ana şalter açıksa bu iki şalter işe yarar; kapalıysa hiçbir şey çalıştırılamaz.

Ana şalter	Işık şalteri	Güç şalteri	Işık	Motor
+	+	+	+	+
+	+	-	+	-
+	-	+	-	+
+	-	-	-	-
-	+	+	-	-
-	+	-	-	-
-	-	+	-	-
-	-	-	-	-

“Şu girdi kombinasyonu olduğunda şu çıktı kombinasyonu oluşur” biçimindeki kesin, değişmez ve birbirini etkilemeyen tanımlara kombinatoryal sistemler denir.

Aynı bilgi karar ağacıyla da gösterilir; daha okunabilir bir gösterimdir:

```
Ana şalter açık mı?
  Hayır -> ışık kapalı, motor kapalı
  Evet  -> Işık şalteri açık mı?
             Evet  -> Güç şalteri açık mı? (Evet: ışık açık, motor açık / Hayır: ışık açık, mo
             Hayır -> Güç şalteri açık mı? (Evet: ışık kapalı, motor açık / Hayır: ikisi de ka
```

Girdi kararları girdinin aldığı değerlere göre dallanır; yapraklarda çıktı vardır. Önceki duruma bağlılık yoktur: belirli bir girdi için belirli bir çıktı ya da durum söz konusudur.

8. Ardışık sistemler: garaj kapısı

İkinci seviyede ardışık sistemler vardır. Bir bellek ya da tarihçe bulunur: sistem önceden belli bir durumdaydı ve olaylar onu bu duruma bağlı olarak yeni bir duruma taşır. Bulunabilecek durumların sayısı sınırlı olduğu için bu tam olarak bir sonlu durum sistemi tanımıdır ve en iyi sonlu durum makineleriyle (finite state machine) modellenir.

Fiziksel bir örnek: bir garaj kapısı. Bir motor kapıyı ya yukarı iter, ya aşağı çeker ya da durur. Motoru çalıştırıp durduran tek bir düğme (on-off) vardır. Düğmeye basılınca kapı sonuna kadar açılıp durur; bunun için “tam açıldı” sensörü gerekir. Ya da tamamen kapanıp durur; kepenk gibi sonuna kadar indiğini algılayan bir sensör vardır. Arada herhangi bir yerde düğmeye basmak, panik butonu gibi kapıyı durdurur: “eyvah, altından kedi geçiyor, sıkışmasın”.

Düğmeye basıldığında olacaklar mevcut duruma göre değişir:

- Kapı ve motor durmuşsa: kapı ters yönde hareket etmelidir.
- Kapı yukarı çıkarken: durmalıdır.
- Kapı aşağı inerken: “eyvah, kedi eziliyor” diye durmalıdır.

Aynı olaya karşı kapı inerken, çıkarken ve dururken farklı davranışlar sergilemelidir. Önceki duruma göre olayın farklı durumlara geçirdiği bu tür sistemlere ardışık sistemler denir.

Durumlar: kapı açık motor kapalı; kapı kapalı motor kapalı; kapı yarım açık motor kapalı; kapı aşağı iniyor motor çalışıyor; kapı yukarı çıkıyor motor çalışıyor; ve panik butonu yüzünden geçici olarak durdurulmuş yarım açık kapı. Olaylar: bir düğme ve iki sensör (kapı açıldı, kapı kapandı), yani üç olay.

Slayttaki durum geçiş tablosu (ekrandan, 00:27:20):

Mevcut durum	Girdi	Eylem	Sonraki durum
Kapı kapalı motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor
Motor yukarı çalışıyor	Kapı açıldı sensörü	Motor durdur	Kapı açık motor duruyor
Motor yukarı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor

Mevcut durum	Girdi	Eylem	Sonraki durum
Kapı açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor
Motor aşağı çalışıyor	Kapı kapandı sensörü	Motor durdur	Kapı kapalı motor duruyor
Motor aşağı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor

Hoca tabloyu okurken "kapı yarım açık motor duruyor" durumuna iki geçiş olduğuna dikkat çeker: birinde motor yukarı, birinde aşağı çalışıyordu. Bu durumdayken düğmeye basılınca motor ya aşağı ya yukarı çalışır. Slaytın alt notu: durum ve olay sayısı arttıkça tablo anlaşılmaz bir hâl alır.

9. Durum geçiş diyagramı ve telefon örneği

Tablo yerine bir **durum geçiş diyagramı** çizilir. Bu grafik bir gösterimdir: durumlar genellikle yuvarlak ya da oval, üzerinde yazı varsa dikdörtgen çizilir. Bağlantılar durum geçişleridir: hangi olayda hangi eylemle hangi duruma geçileceği yazılır.

> **Hocanın vurgusu:** Düğümlerin nerede durduğunun hiçbir anlamı yoktur. Diyagramı 90 ya da 180 derece çevirseniz de aynıdır; "en yukarıda duruyor" gibi bir anlam yoktur.

Herhangi bir anda belli bir durumdayızdır; bir olay olana kadar o durumda kalırız. Olay olduğunda belli bir eylemle geçiş yaparız.

Epsilon geçiş: Hiçbir girdi almadan da geçiş olabilir. Hoca garaj kapısında güvenlik nedeniyle, hiçbir uyaran olmadan bir durumdayken motoru çalıştırıp başka bir duruma geçildiğini örnek verir. Olay dışında yapılan bu otomatik durum geçişlerine epsilon geçiş denir.

9.1 Telefon örneği

Slayttaki diyagram (ekrandan, 00:30:00):

- Durumlar: telefon kapalı (ON HOOK, çift çizgili, sistemin başlangıç durumu), telefon açık (OFF HOOK), tuşlanıyor (DIALING; kendine geçişli, 1., 2., … numara çevriliyor, her aşama ayrı durum), çalıyor (RINGING), meşgul (BUSY), bağlı (CONNECTED).
- Geçişler:

```
ON HOOK   --Handset lifted-->      OFF HOOK
OFF HOOK  --Number dialed-->       DIALING
DIALING   --Number dialed-->       DIALING (kendine)
DIALING   --Last number dialed-->  RINGING
RINGING   --Line in use-->         BUSY
RINGING   --Callee answers-->      CONNECTED
CONNECTED --Callee hangs up-->     ON HOOK
OFF HOOK, DIALING, RINGING, BUSY, CONNECTED --Handset replaced--> ON HOOK
```

Hoca akışı anlatır: ahize kalkınca telefon açık durumuna geçilir; buradan ahize bırakılabilir ya da numara çevrilebilir. Numaraya basıldıkça tuşlama durumunda kalınır. Son numara çevrilince (burada bir kontrol de olmalıdır) çalma durumuna geçilir. Çalma durumundan üç şey olabilir: telefon kapatılabilir, meşgul tonu alınabilir ya da hat açılabilir. Hat açıldıktan sonra arayan ya da aranan kapatabilir; sonuç kapalı durumdur.

Slayttaki not: diyagramda gereksiz etiketli geçişler vardır; eş zamanlı sistemler modellenirken bu diyagram basitleştirilerek iyileştirilecektir.

9.2 Durum geçiş diyagramlarının sorunları

- Çok fazla geçiş oluşabilir: n durum ve m olay varsa çarpımları mertebesinde olası geçiş vardır. Hepsi bulunmak zorunda değildir ama uzay bu kadar büyüyebilir.
- Genellikle çok fazla durum vardır ve olabileceklere göre uzay yine üstel artar.
- İç içe bulunma ifade edilemez. Telefon örneğinde kaç numara çevrilmesi gerektiği, döngünün kaç kez döneceği gibi bir kontrol gösterilemez.

Bu sorunları en azından kısmen çözmek için durum diyagramları (statecharts) kullanılacaktır. Hoca, bazen statecharts'ın bile yetmeyeceği karmaşıklıkta sistemlerle karşılaşılabileceğini de ekler.

10. Durum diyagramları (statecharts)

Statecharts'ı David Harel geliştirmiştir. Sonlu durum makinesini, yani durum geçiş diyagramını iyileştirmiş ve buna durum diyagramı adını vermiştir. Temel katkısı karmaşıklığı yönetmek için çeşitli mekanizmalar tasarlamaktır. Bir mimar, özellikle sistemin vereceği tepkileri modellerken UML'de bunu kullanabilir; UML'de statechart vardır ve CASE araçları bunu destekler.

Semboller basittir:

- Durumlar kenarları yuvarlatılmış dörtgenlerle çizilir. Örneğin bir boş (idle) ve bir aktif durum olabilir; aktif durumdan idle duruma belirli bir olayla, belirli bir eylem yapılarak geçilir.
- Son durum (final state), içi dolu bir noktanın çevresine bir çerçeve daha çekilerek gösterilir. Olaylar olup bittiğinde sistemin son durumunu ifade eder.

Durum diyagramlarının durum geçiş diyagramlarına eklediği özellikler:

- **Derinlik (iç içe gruplama):** bir durum kendisi de karmaşık bir durum makinesi olabilir; içine girip zoom yapmak gibi düşünülebilir.
- **Eş zamanlılık:** en önemli katkılardan biridir ve durum uzayını çok azaltır. Normalde $M \times N$ olan karmaşıklık, olaylar ayrı ayrı ele alındığı için $M + N$ mertebesine iner.
- **Koşullu geçişler:** geçiş yalnız belli şartlarda gerçekleşir.
- **Giriş-çıkış eylemleri ve aktiviteler.**
- **Olay parametreleri:** olay sayesinde veri aktarılabilir; hoca bunu önemli bulur.
- **Hatırlama (history):** hangi alt durumda kalındığını hatırlama.
- **Varsayılan durum:** bir şey söylenmediyse sistemin hangi durumda olması beklendiği.

11. İç içe gruplama

11.1 Soyut örnek: A, B, C ve D

Slayttaki iki diyagram birbirinin birebir eşleniğidir (ekrandan, 00:35:00):

```
Sol (düz):
  başlangıç -> A
  A --g(c)--> C
  A --f-->    B
  C --f-->    B
  B --e-->    A
  B --h-->    C

Sağ (gruplanmış):
  başlangıç -> D, D'nin varsayılanı A
  D = { A, C },  A --g(c)--> C
  D --f-->  B          (A'dan ve C'den çıkan iki f oku yerine tek ok)
  B --e-->  A
  B --h-->  D          (D'ye bu geçişle girilince varsayılan C)
```

Solda A'da da olsak C'de de olsak `f` olayıyla B'ye geçeriz. Sağda A ve C, D adlı bir grupta toplanır ve D'den tek bir `f` çıkarılır. B'den `e` ile A'ya geçilir; `h` ile ise yalnız D durumuna geçilir.

Hoca bunu şöyle açıklar: dışarıdan başlarken D'nin varsayılanı A'dır, ama D'ye `h` geçişiyle girildiyse C'ye gelinir; soldaki `h` oku da C'ye gidiyordu. Bu çok basit örnekte doğrudan çizmek de bir şey değiştirmezdi; amaç toplam giriş-çıkış, yani durum geçiş sayısını azaltmaktır.

11.2 Isıtma-soğutma örneği

Hoca bir iklimlendirme sistemi örneği anlatır. Sistem normalde Idle (bekleme) durumundadır; shutdown edilirse son duruma geçer.

- Çok sıcak olursa (too hot) Cooling durumuna geçilir; burada istenen sıcaklık (desired temp) parametre olarak verilebilir. Klima ortamı soğutur; o sıcaklığa gelinince yeniden Idle'a dönülür.
- Çok soğuk olursa (too cold) Heating durumuna geçilir. Heating bir sanal gruptur: içinde Activating (varsayılan) ve Active durumları vardır; hazır olunca (ready) Active'e geçilir.
- Activating'de de olsak Active'de de olsak çok ısınırsa Idle'a uğramadan doğrudan soğutmaya geçilebilir; Cooling'deyken çok soğursa doğrudan ısıtmaya geçilebilir.

Hoca ısıtma ile soğutma arasında doğrudan geçişin biraz garip bir davranış olduğunu kabul eder; amaç, her iç durumdan ikişer ok çıkarmak yerine gruplayarak daha az ok ve daha az geçiş çizmektir.

11.3 Printer örneği

Slayttaki "İç İçe Gruplama Örneği" (ekrandan, 00:40:00):

- Üç üst durum vardır: idle, maintenance ve active.
- Active içinde dört durum vardır: Validating, Selecting, Processing ve Printing. Idle'dan gelen geçişler varsayılan olarak Validating'e bağlanır.
- `[continue]` ve `[not continue]` koşullu geçişleri: bu durum makinesinde temsil edilen nesnenin özellikleri üzerinde mantıksal test ile sağlanır.
- Entry ve exit ile belirlenen eylemler, idle durumundan gelen ve giden geçişlerde alınacak aksiyonları ifade eder: hiçbir şey yapılmadan önce `readCard` yapılır; active durumundan çıkılırken `ejectCard` yapılır.

```
başlangıç -> Idle
Idle --cardInserted--> Active
Idle --cancel-->       Active
Idle --maintain-->     Maintenance --> Idle
Active (entry / readCard, exit / ejectCard):
  başlangıç -> Validating -> Selecting -> Processing
  Processing --[continue]-->     Selecting
  Processing --[not continue]--> Printing
  Printing --> Idle
```

Hoca koşulu şöyle okur: Processing'de `continue` doğruysa sürekli Selecting'e, oradan Processing'e geçilir. `not continue` doğru olduğunda, yani nesnenin taşıdığı bu Boolean özellik false olduğunda Printing'e geçilir. Entry ifadesinin anlamı: Active kutusuna ister `cardInserted` ister `cancel` ile girilsin, Validating'e geçmeden önce mutlaka kart okuma eylemi yapılır. Exit ifadesinin anlamı: bu durumdan herhangi bir yerden, nasıl olursa olsun çıkılıyorsa (diyagramda yalnız Printing'den çıkış gösterilmiştir) mutlaka `ejectCard` davranışı çalıştırılır.

12. Eş zamanlılık

Durum diyagramlarının en güzel yanlarından biri eş zamanlılıktır. Bir durumun aynı anda çalışan parçaları varsa bunlar kesikli çizgiyle ayrılarak gösterilir. Her makinede mevcut durumlar vardır; eş zamanlı makineler aynı olaylara tepki vererek durum geçişleri yapabilir ve her biri geçişlerde çeşitli aksiyonlar yürütebilir.

12.1 Y durumu örneği

Slayttaki iki diyagram aynı davranışı anlatır (ekrandan, 00:41:30). Solda düzleştirilmiş hâlde B,E; B,F; B,G; C,E; C,F; C,G; H ve I durumları ile `k`, `g`, `h`, `e`, `f`, `p`, `n`, `m` olaylarının çok sayıda oku

vardır. Sağda ise:

```
Y = A || D   (kesikli çizgiyle ayrılmış iki eş zamanlı parça)
  A: B (varsayılan), C
     B --e--> C
     C --f[in G]--> B
  D: E, F (varsayılan), G
     F --k--> E,  E --g--> G,  G --h--> F,  F --e--> G
  D --m--> I   (F'den)
  Y --p--> I
  I --e--> C ve G   (aynı anda iki parçaya geçiş)
  A --n--> H   (C'den)
```

Hoca okur: Y durumundayız ve e olayı oluştu. Eş zamanlı parçalardan A, bir alt sistem ya da modül gibi düşünülebilir; B'den C'ye geçer. Aynı anda D parçasında F'deysek G'ye geçeriz; E'deysek e ile bir şey olmaz, E'de kalırız. I durumundayken e olunca da doğrudan G ve C durumlarına geçiş modellenebilir. Böylece karmaşa daha okunabilir ve takip edilebilir hâle gelir.

12.2 UML'de eş zamanlılık

UML diyagramında da eş zamanlılık kesikli çizgiyle gösterilir. Hocanın örneğinde bir Maintenance durumu vardır; içinde iki eş zamanlı durum makinesi bulunur: Testing alt durumu ve Commanding alt durumu.

- Testing içinde Testing Devices, Self Diagnosis ve son durum arasında otomatik geçişler yapılır.
- Commanding içinde beklenir; bir tuşa basılınca Command durumuna geçilir; devam edildikçe (`continue`) Waiting'e dönülür, devam edilmezse (`not continue`) Commanding'den çıkılır.

İki parça da son durumlarına geldiğinde Idle durumuna dönülür. Biri bitmiş ama diğeri bitmemişse, son durumdaki parça bekler; ne zaman ki `not continue` olur ve ikisi de son durumuna erişir, o zaman Idle'a geçilir. Eş zamanlılık bu tür kontrolleri de sağlar.

13. Senkronizasyon: yayım (ardarda) olaylar

Eş zamanlı makineler her zaman bu kadar bağımsız değildir. Hoca, birbirleriyle hiçbir beklentisi olmayan iki makinenin (bir önceki örnekteki Testing ve Commanding gibi) ayrı ayrı modellenmesinin de bir şey değiştirmeyeceğini söyler; alt makinelerin bir arada konmasının anlamı, birlikte çalışmalarının beklenmesidir. Bu durumda koordine ya da senkronize edilmeleri gerekir. Bunun iki yolu vardır: yayım ya da ardarda olaylar (broadcast cascade) ve veri durumuna göre koşullu geçişler.

Slayttaki örnek (ekrandan, 00:46:00):

```
A || D || H   (üç eş zamanlı makine)
  A: B (varsayılan) --f/g--> C,  C --e--> B
  D: F (varsayılan) --k--> E,  F --e--> G,  E --g--> G,  G --n--> E
  H: J (varsayılan) --m/e--> I,  I --n/f--> J
```

Slayttaki açıklama: A, D ve H üç eş zamanlı makinedir. Aralarında geçişler, bazılarında aksiyonlar vardır (nesneye yönelik sınıf diyagramlarındaki metot çağrıları gibi). Fakat aksiyon başka bir olayın başlaması da olabilir. Olaylar genel görünürdür; bu yüzden eş zamanlı makineler de bunları bilir.

Hoca örneği okur: A makinesinde B durumundayız ve `f` olayıyla C'ye geçiyoruz. `f/g`, `f` olayı olduğunda aynı zamanda bir `g` olayının da yaratıldığı, tetiklendiği anlamına gelir. Eş zamanlı D makinesi E durumundaysa `g` ile G'ye geçer. F'deyse `g` olduğunda hiçbir şey yapılmaz. Böylece olaylar arka arkaya konmuş olur.

İkinci örnek: H makinesinde `m` olayı oldu. `m/e`, `e` olayının olmasını yaratır. A makinesinde C'deysek ve D makinesinde F'deysek, `e` iki ayrı durum geçişini aynı anda tetikler. Bu yapıya yayım ya da ardarda olaylar (broadcast cascade) denir.

Ders sonrası not: Slayt metni "A'da C'den B'ye F geçişi var" der; diyagramdaki `f/g` oku ise B'den C'ye gider. Hoca da derste "B'den C'ye ters yazmışım" diyerek bu tutarsızlığa değindi.

14. Koşullu ve özel geçişler

Koşullu geçişler daha önce `continue` ve `not continue` örneğinde görüldü. Bir verinin aldığı değerlere göre köşeli parantez içinde koşul belirtilir. Bunlar genellikle Boolean'dır (true ya da not true); sürekli izlenir ve doğru oldukları anda ilgili geçiş gerçekleşir. `in` ve `not in` de kullanılabilir: örneğin başka bir eş zamanlı makine X durumundaysa ancak bu geçiş yapılır denebilir. Bu koşullar da bütün eş zamanlı makinelere görünür.

Hoca bir zaman koşulu örneği verir: `when 11.49 / self-test`; her gün saat 11.49'da Idle durumunda bir self-test çalıştırılıp yeniden Idle'a gelinir. Bir başka örnekte Active durumunda yalnız iki saniye kalınmasına izin veren bir sayaç vardır; süre dolunca bağlantı düşürülerek (drop connection) Idle'a dönülür. Hoca bu örnekte Idle'dan Active'e bir geçiş tanımlanmadığını, normalde böyle bir geçişin de beklendiğini belirtir.

14.1 Özel geçişler: Idle-Tracking örneği

Slayttaki diyagram (ekrandan, 00:50:00):

```
başlangıç -> Idle
Idle --after: 2 seconds / send c.isAlive--> Idle   (kendine geçiş)
Idle --noise--> Searching --> Engaging --> Tracking
Idle --targetAt(p)[isThreat] / t.addTarget(p)--> Tracking
Tracking --contact--> Engaged --> son durum
```

Slayttaki açıklama: Idle durumunda kendine geçiş vardır; diğer durumlarda normal geçişler bulunur. Idle ile Tracking arasındaki geçişte bir eylem vardır: bir metodu tetikler ve metodun `p` parametresi vardır. Eylem çağrılarında bilgi aktarılabilir; geçişe neden olan olayda da `p` parametresi vardı, bilgi aktarılmış oldu.

Hoca okur: Idle'da iki saniyede bir `c.isAlive` gönderilir; "yaşıyorum" diye bir heartbeat verilir ama Idle'da kalmaya devam edilir. `targetAt(p)` olayı geldiğinde ve `isThreat` (bir tehdit var) doğruysa `t` nesnesinin `addTarget` metodu çalışır ve `p` parametresi ona aktarılır. Böylece gelen bilgi başka bir nesneye aktarılmış olur. Durum geçişlerinde parametrelerle bilgi aktarımı bu şekilde sağlanır. Idle'da gürültü (noise) olursa Searching'e geçilip aranır, sonra Engaging'e, oradan Tracking'e geçilir. Tracking'den Idle'a dönülmez; yalnız temas (contact) olduğunda Engaged'e geçilir, oradaki işler bitince son duruma varılır.

15. Geçmiş (history)

Geçmiş durumları hatırlama özelliği history ile sağlanır. Hocanın örneğinde Command ve Backup durumları arasında geçiş yapılabilir. Backup'ın içinde Collecting, Copying ve CleaningUp gibi durumlar vardır. Backup içindeki herhangi bir durumdan bir sorgu (query) ile Command'a çıkılmış olabilir; sonra yeniden Backup'a geçilir. Bu geçişte en son nereden çıkıldığını hatırlamak gerekiyorsa bir `H` konur. Örneğin Copying'deyken sorgu ile çıkıldıysa, Backup'a dönüldüğünde Copying'den başlanır.

`H` yerine `H*` de konabilir. O zaman iç içe makineler varsa onların içindeki geçmiş durumlara da dönülür. Hoca bunun oldukça güçlü bir mekanizma olduğunu, ama okunurluğunun ve takibinin zorlaştığını söyler: Collecting'in ya da Copying'in içini de history ile takip etmek gerekecektir.

16. UML'de durum ve geçiş tanımı

Slayttaki UML durum tanımı (ekrandan, 00:53:00):

- En üstte durumun adı.
- Girdi ve çıktıda, varsa, alınacak aksiyonlar: `entry`, `exit`.
- Girdi-çıktı işlemleri olmadan kendine geçiş: `newTarget`.

- Aksiyonlar aç/kapa gibi zaman almayan eylemler, aktiviteler ise zaman alan eylemlerdir: `do`.
- Kuyruğa eklenip daha sonra işletilen eylemler: `defer`.

Slayttaki örnek:

```
Tracking
  entry / setMode(onTrack)
  exit / setMode(offTrack)
  newTarget / tracker.Acquire()
  do / followTarget
  selfTest / defer
```

Hoca açıklar: bu duruma girerken mutlaka çalışması ya da çıkarken mutlaka çalışması gereken şeyler `entry` ve `exit` ile belirtilir. Girdi ya da çıktı olmadan kendi üzerine geçiş yapmak, yani otomatik bir metot çalıştırmak için `newTarget` gibi bir tanım kullanılır. Kuyruğa atılıp sonra bir ara yapılabilecek, çalışması ötelenebilecek eylemler `defer` ile verilir. Hoca, çoğu modellemede yalnız durumun adını ve varsa entry-exit'leri yazmanın yeterli olacağını söyler.

> **Ders sonrası not:** Hoca anlatırken `do`'yu önce "aç, kapa gibi zaman almayan eylemler" için söyledi. Slayta göre zaman almayan eylemler aksiyonlardır; `do` zaman alan aktiviteler içindir.

Bir geçiş tanımlanırken şunlar belirtilir:

- Kaynak durum: geçişe nereden gelindi?
- Hedef durum: nereye gidilecek?
- Tetikleyen olay.
- Üzerinde kontrol (guard) var mı? Örneğin `continue`, `not continue`. Hoca, geçen hafta söylediği gibi bunların OCL ile tanımlanması gerektiğini hatırlatır.
- Geçiş sırasında yapılacak bir aksiyon, örneğin çalışacak bir metot.
- Ayrılmalar ya da birleşmeler: örneğin tek bir eylem iki farklı eş zamanlı makinede durumları tetikleyecek mi?

17. Sınıf diyagramı ile ilişki

Durum diyagramı çizilirken UML'in yapısal modeliyle nasıl bir ilişki olduğu sorusu sorulur. Durum geçiş diyagramları oldukça güçlüdür ve teorik olarak her sınıfa ayrı bir durum diyagramı çizilebilir. Fakat bu da durum patlaması yaratabilir.

Genellikle sınıfların çoğu için bu kadar durum geçişi tanımlamaya gerek yoktur; "şu olduğunda şu metot çalışsın" biçimindeki metotlar (get, set gibi) yeterlidir. Belirli uyaranlara, özellikle dış uyaranlara göre belirli metotların çalışmasını ya da özelliklerin belirli değerlerde olmasını göstermek veya garanti etmek gereken karmaşık modellemelerde durum diyagramı kullanmak doğrudur. Pratikte yalnız ihtiyaç olan yerde yapmak yeterlidir.

18. Casio F-91W dijital saat örneği

Hoca durum diyagramının gücünü göstermek için Harel'in modellediği Casio F-91W dijital saatinin durum diyagramını gösterir (ekrandan, 01:00:00).

18.1 Üst seviye

```
dead (varsayılan) --bt-in--> alive
alive --bt-rm/clr--> dead
alive (eş zamanlı parçalar):
  main    : displays --t-hits-tm [in(enab)]--> beep,  beep --beep-rt--> displays
  power   : OK (varsayılan) --bt-wk--> weak,  weak --bt-dy--> dead
  light   : off (varsayılan) --b--> on,  on --b-up--> off
  alarm-st: disab (varsayılan) <--d[in(alarm)]--> enab
  chime-st: c-disap (varsayılan) <--d[in(chime)]--> c-enab
            c-enab: quiet <--t-hits-hr / beep-st--> c-beep
```

Varsayılan olarak dead durumundayız; pil yoktur. Pil takılınca (battery in) alive durumuna geçilir. Alive'dayken pil çıkarılırsa temizlenerek (clear) dead'e geçilir. Power alt makinesinde normalde OK durumundayız; pil zayıflayınca weak'e, pil ölünce dead'e geçilir.

Pil takılıp alive'a geçildiğinde aynı anda main, power, light, chime ve alarm makinelerinin hepsi, beş eş zamanlı makine çalışmaya başlar. Light'ta varsayılan off, power'da OK, alarm'da disabled'dır. Main'de display çalışır; saat başlarında bip bip öten beep moduna geçilip geri gelinebilir.

18.2 Displays ve stopwatch

Display kendi içinde bir durum makinesidir; varsayılanı zamanı göstermektir. A, B, C, D saatin üzerindeki dört düğmedir:

- D'ye basılınca tarih gösterilir, tekrar D'ye basılınca zamana dönülür.
- A'ya basılınca alarm saati gösterilir; B, C gibi düğmelerle ayarlanır.
- A'ya bir kez daha basılınca saat başı çalsın mı çalmasın mı (chime) ekranı, bir daha basılınca stopwatch ekranı, bir daha basılınca yeniden zaman gelir. Hoca, saati kullananların mod tuşuna dört kez basarak saate döndüğünü hatırlatır.
- Başka bir tuşla ayar moduna geçilir; düğmeye her basışta sırayla dakika, saat, tarih, gün gibi alanlar ayarlanır, en sonunda çıkılıp yeniden zaman gösterilir.

Stopwatch'a gelindiğinde varsayılan olarak sıfırdadır. İçinde display ve run kısımları eş zamanlı çalışır; sayaç ve tur zamanları vardır. Stopwatch başlatılıp zaman ekranına dönülse bile stopwatch kendi içinde çalışmaya devam eder. Chime ekranından yeniden stopwatch'a gelindiğinde history (`H*`) sayesinde kalınan yerden devam edilir: örneğin 4 dakika 26 saniye 54 salisede kalındıysa oradan göstermeye ve çalışmaya devam eder.

Diyagramda basitlik için bu parçalar soyutlanmış, içleri ayrı ayrı gösterilmiştir; hepsi iç içe konduğunda dört düğmeli bu saatin pil ve düğmelerle ilgili bütün durum geçişleri ortaya çıkar. Koşullar da gösterilir: bip ayarlandıysa şuraya, ayarlanmadıysa buraya; alarm kuruluysa şuraya, değilse bu tarafa geç gibi.

19. Ara özet: reaktif sistemleri modellemek

Reaktif sistemler uyaranlara tepki veren sistemlerdir. Bunları inşa etmek ilk aşamada daha zordur; karmaşık sistemler bazen kilit durumlara girer. Durum diyagramları bu karmaşanın yönetilmesini ve durum geçişlerinin düzgün modellenmesini sağlar. Diğer davranışsal diyagramlar (aktivite, ardışıl, işbirliği, kullanım senaryosu; iletişim, timing ve etkileşim genel bakış diyagramları zaten diğerlerinden oluşur) da kullanılabilirdi.

Soru: UML'de statechart kullanmazsak reaktif ya da olay tabanlı bir sistemi tanımlarken ne kullanabiliriz? **Cevap:** Temporal logic ya da process algebra gibi biçimsel yöntemlerle de benzer modellemeler yapılabilir.

Hoca yine vurgular: bütün sınıfların kendine ait birer durum diyagramı olabilir, ama hepsini yapmaya gerek yoktur. Karmaşık yerler ve kilit durumların oluşabileceği yerler ile sistemin bunlara vereceği tepkiler modellenmelidir; o zaman durum diyagramı kullanmak gerekir.

20. Radyo saat örneği

Aradan sonra hoca, durum geçişlerinin nasıl tasarlandığını ve değişkenlerin aldığı değerlere göre hangi olayların nasıl yer aldığını göstermek için çok basit bir sistem örneği yapar: eskiden yaygın olan bir radyo saat.

20.1 Sistemin tanımı

- Saati gösteren dijital bir ekran; 12 saatlik zaman.
- Analog bir radyo; yan taraftaki çarkla frekans (tuning) ayarı yapılır, frekansı gösteren beyaz bir çubuk vardır.
- Sesi artırıp azaltan bir ses düğmesi (volume knob).

- İki küçük lamba: biri AM/PM'i (öğleden önce/sonra), diğeri alarmın kurulu olup olmadığını gösterir.
- İki şalter: biri AM/FM bandını seçer; diğeri dört konumludur: radyo kapalı, radyo açık, alarm radyo ile, alarm bipleme ile.
- Soldaki dört düğme: uyanma (wake) ve uyuma (sleep) saatlerini ayarlamak için saat ve dakika düğmeleri. Belirli bir saatte radyonun kapanması, belirli bir saatte radyonun ya da alarmın çalması ayarlanabilir.
- Bir hoparlör.
- Elektrikle çalışır, pili yoktur; bu yüzden Casio saatteki gibi pil zayıfladı, pil öldü makinesine gerek yoktur.

Hoca bir ayar örneği verir: şu an saat 11.20 olsun ve yarın 11.35'te uyanmak isteyelim. Wake butonu basılı tutularak dakika düğmesine basılır ve saat kurulur.

- Snooze butonuna her basıldığında alarm 10 dakika ertelenir: 10 dakika susar, sonra yeniden çalmaya başlar.
- Her hâlükârda alarm bir saat çaldıktan sonra otomatik olarak kapanır. Hoca bunu "bir saatte uyandıramadıysa komşulara da yazık" diye açıklar.

Durum diyagramından önce kullanım senaryolarına bakmak gerekir: zamanı ayarlama, uyanma ve uyuma saatini ayarlama, frekans değiştirme, snooze'a basma, radyo dinleme gibi. Durum diyagramı bu senaryolardaki bütün durum geçişlerini kapsamalıdır. Beklenmedik bir şey olduğunda radyonun ne yapacağı gibi hata durumları da eklenebilir.

20.2 Perceptler

Durum geçişlerini modellemek için perceptleri, yani kullanıcının o anda görebildiği ve etkileşebildiği özellikleri bilmek gerekir. Radyo saatin perceptleri:

- Zaman göstergesi.
- İki ışık: AM/PM ve alarm kurulu mu.
- Zamanlayıcı düğmeleri: belirli bir durumları yoktur ama basma ve çekme olayları vardır.
- Frekans ayar düğmesi ve onunla hareket eden beyaz çubuk.
- Çevirmeli ses düğmesi.
- AM/FM şalteri.
- Dört konumlu mod şalteri.
- Snooze butonu.
- Hoparlör.

Hoca, bir sistemi modellerken bu perceptlerden yola çıkmak gerektiğini söyler.

20.3 Zaman göstergesi kaç durumda olabilir?

Slayttaki hesap (ekrandan, 01:30:00):

- Verilen: iki ışık şimdilik düşünülmez; saatler 01..12, dakikalar 00..59 arasıdır.
- İşlem: 12×60.
- Sonuç: 720 durum. 11.27, 11.28, 11.29 gibi her biri ekran açısından ayrı bir durumdur. Bu, 720 durumlu bir sonlu durum makinesiyle modellenebilir.
- Durum diyagramında saat ve dakika için iki eş zamanlı makine kullanılırsa $12 + 60 = 72$ durumda anlatılabilir.
- Bu bile fazladır. "Saat" adlı tek bir düğüme soyutlanır ve 720 durumun beklendiği şekilde, her dakika bir sonrakine geçerek oluştuğu varsayılır.

Gösterge eş zamanlı bir alt makine (Display) olarak çalışır ve üç durumu vardır: saati gösteren ClockTime, uyanma saatini gösteren WakeTime ve kapanma (uyku) saatini gösteren SleepTime. Ayar moduna geçildiğinde başka bir makinedeki olayla burada da durum geçişi yaşanır: wake butonu basılıyken ayarlanan saat ekranda görülür (WakeTime); sleep'e basıp saat ayar düğmesine basınca SleepTime görülür.

> **Hocanın vurgusu:** Oklar iki yönlü çizilmiştir ama aslında her okta iki ayrı geçiş vardır: sleep düğmesi basıldığında bir yöne, bırakıldığında öbür yöne geçilir.

20.4 Mod şalteri ve diğer eş zamanlı makineler

Mod şalterinin dört konumu vardır: kapalı, radyo açık, müzikli (radyolu) alarm ve bipli alarm. Radyo saatte bu dört konum yan yana dizilmiştir; şalter "çıt çıt" kaydırılarak yalnız yanındaki konuma geçebilir. Örneğin On durumundan doğrudan Alarm durumuna geçilemez. Bu yüzden Mode makinesinde yalnız komşu konumlar arasında geçiş vardır (ekrandan, 01:40:00):

```
Mode: On <-> Off <-> Music <-> Alarm
      (12: mod düğmesini sağa kaydır, 13: sola kaydır)
```

Band şalteri de ayrı bir eş zamanlı makinedir: AM'deyken FM'e, FM'deyken AM'e geçilir. Hoca bunların neden eş zamanlı olduğunu açıklar: herhangi bir anda AM/FM değiştirilebildiği gibi aynı anda öbür şalter de değiştirilebilir. Bu şalterlerle oynanırken ekran saati göstermeye devam eder; sistemin başka bir davranışı da eş zamanlı çalışır.

Diğer makineler:

- **Station (istasyon göstergesi):** çark çevrildikçe beyaz çubuk kayar; sonsuz sayıda konumda olabilecek bu yapı kendine dönen bir makineyle gösterilir.
- **VolumeDial:** ses çarkı çevrilince yeni bir ses durumu oluşur; bu da kendine dönen bir eş zamanlı makinedir.
- **Station'ın frekansı:** radyonun belli bir frekansta (bilmem kaç MHz ya da kHz) olduğu bir durumu vardır; AM ya da FM'de olmasıyla da değişir. Çark değiştikçe bu da değişir; bir olay, algılanan frekans değerinin değişmesine yol açar.
- **Speaker (hoparlör):** elektronik ayrıntıya girilirse direnç, gerilim gibi pek çok durum olabilir, ama yazılım açısından hoparlörün açık ve kapalı olması şimdilik yeterlidir. Diyagramda Silent, Playing ve Beeping durumları vardır.

Hoca, sistemle etkileşilen bütün noktaları birer eş zamanlı durum geçişi alt makinesi olarak modellediğini söyler. Bu noktada yedi makine yapılmıştır ve henüz geçişler yoktur.

20.5 Olaylar

Sonraki adım, durum geçişlerini oluşturan olaylardır: sistem harici olaylara nasıl tepki verir?

- Şalter sola ya da sağa kaydırılır. Bunlar iki ayrı olay olarak ya da yönü parametre olan tek bir kaydırma olayı olarak modellenebilir; tasarımcıya kalmıştır.
- Çevrilen düğmelerde bir açı değeri parametre olabilir.
- Snooze butonu yalnız basılabilir (press); kullanıcı açısından başka bir şey yapılamaz.

Hoca, alarm çalmazken snooze'a basılınca sistemin nasıl tepki vereceğine tasarımcının karar vermesi ve bunu durum diyagramıyla söylemesi gerektiğini belirtir. Örneğin saat 11.30'dayken alarm 12'ye kurulu ve snooze'a iki kez basıldıysa alarmın 12.20'de çalması istenebilir; ya da alarm çalmaya başlamadan snooze'a basmanın hiçbir etkisi olmaması istenebilir. Bunlar tamamen sistemin nasıl çalışacağına dair verilen kararlara bağlıdır ve durum diyagramı bunları anlatır.

Uyanma ve uyuma düğmelerine, saat ve dakika düğmelerine basılır; bu eylemler olay yaratır. Slaytta analiz sırasında 17 olay belirlenmiştir.

En dış durum: Casio saatte pil takılı, pil ölü ya da pil yok durumları vardı. Burada fişe takılı (plugged) ya da fişi çekilmiş (unplugged) durumları vardır. Fiş takılınca plugged'a, çekilince unplugged'a geçilir. Bütün eş zamanlı makineler plugged durumunun içinde çalışır.

Olaylar diyagramın üstüne yazılabilir; ama çok olay varsa karışabileceği için bir liste yapılır ve olaylar numaralandırılarak makineye yerleştirilir. Slayttaki liste (ekrandan, 01:38:30):

- 1. Ses ayar düğmesini çevir.
- 2. AM bandını seç.
- 3. FM bandını seç.
- 4. İstasyon seç.
- 12. Mod seçim düğmesini sağa kaydır.
- 13. Mod seçim düğmesini sola kaydır.

Snooze ile ilgili başka bir makineye de ihtiyaç vardır: her basışta alarm 10 dakika susar, sonra yeniden çalar. Bu kullanıcıya görünen bir şey değildir ama kullanıcıyı etkiler; bu yüzden 10 dakika sayıp alarmı yeniden tetikleyecek bir iç sayaç gerekir.

Uyku ve uyanma saati düğmelerine basıldığında da yeni bir duruma girilir; ekran değişir ve o saat görülür. Bunun için **Setting** adlı yeni bir eş zamanlı alt makine kurulur: None (ekranda saat gösterilir), ClockSet, SleepSet ve WakeSet durumları; aralarındaki geçişler 8, 9, 10 ve 11 numaralı olaylarla olur. Hoca, ClockSet için henüz bir şey konuşulmadığını, büyük ihtimalle saat ve dakika düğmelerine doğrudan basıldığında saati ayarlamak için bir geçiş olması gerektiğini söyler. Saat düğmesinin (HrSet) ve dakika düğmesinin (MinSet) up ve down durumları vardır; 15-16 ve 17-18 numaralı olaylarla geçilir.

20.6 Tepkiler ve iç olaylar

Bütün olayların sistemin durumunda bir etkisi olmalıdır. Örneğin ses düğmesi çevrilince hoparlörün sesinin artması beklenir. Mod seçim düğmesi sağa kaydırılınca açıksa kapalıya, kapalıysa müziğe, müzikse alarma geçilir ya da tam tersi yapılır. Uyarılara karşı tepkiler “şu durumdayım, şuraya geç, şunu yap” biçiminde bir tabloyla söylenebilir.

Belirli bir saate gelindiğinde ya biplenecek ya radyo çalacak, ya snooze başlayacak ya da çalma başlayacaktır; bunun için hoparlörün açılması gerekir. Demek ki iç durumlar olmalıdır. Özellikle zamana bağlı olay tetikleyicileri (timer’lar) iç durum olarak modellenir:

- **Snooze sayacı:** snooze’a basılınca başlar, 10’a kadar sayar; 10 dakika sonra alarmın yeniden çalmasını sağlar.
- **Alarm sayacı:** alarm çalmaya başlayınca başlar; 60 dakika sonra ne olursa olsun alarmı kapatır.
- **Uyku sayacı:** başlar ve ayarlanan zamana kadar sayar; susma için en fazla 60 dakika.
- **Saat sayacı (clock timer):** her dakika bir “tik” üretir; her tik ekranda 720 durumdan bir sonrakine, yeni bir saat-dakika ikilisine geçişi sağlar.

Bütün bunlar eş zamanlıdır ve birbirini etkiler. Hoca, etkileşilen her nokta için anlatılması gereken durum geçişlerini eş zamanlı makineler olarak tek bir durum diyagramında topladığını söyler.

Slayttaki diğer iç olaylar (ekrandan, 01:43:50):

#	Uyarı	Tepki
19	Alarm sayacı bitti	If not in Hoparlör:Sessiz go to Hoparlör:Sessiz
20	Zaman, alarm saatine geldi	If in Mod:Muzik go to Hoparlör:Çalıyor; If in Mod:alarm go to Hoparlör:Bipliyor
21	Uyuklama sayacı bitti	If in Mod:Muzik go to Hoparlör:Çalıyor; If in Mod:alarm go to Hoparlör:Bipliyor
22	Zaman, uyanma saati +1 saate geldi	If in Hoparlör:Çalıyor go to Hoparlör:Sessiz; If in Hoparlör:Bipliyor go to Hoparlör:Sessiz

Hoca tabloyu okurken 20 ve 21 numaralı satırların aynı olduğunu fark eder, kopyala-yapıştır hatası yaptığını ve uyuklama sayacı bitince hoparlörün sessiz duruma geçmesi gerektiğini söyler; düzeltmeyi kayıtta fareyle göstererek yapar.

> **Ders sonrası not:** Aynı derste snooze sayacının 10 dakika sonra alarmı yeniden çaldırdığı anlatıldı ve genel diyagramda snooze sayacı bitince hoparlör bipliyor. Bu nedenle slayttaki 21. satır büyük olasılıkla snooze (uyuklama) için doğrudur. Sessize geçme tepkisi büyük olasılıkla uyku (sleep) sayacının bitmesine aittir.

Uyanma saati artı bir saate gelindiğinde de hoparlör sessize alınır. Hoca, iç olayların ve zamanlayıcı olaylarının mutlaka gösterilmesi ve modellenmesi gerektiğini söyler.

20.7 Kontrollü geçiş ve yayım

Diyagramda bir kontrollü geçiş vardır: 20 numaralı olay (zaman alarm saatine geldi) oluştuğunda, müzik modundaysak hoparlör Silent'tan Playing'e geçer. Bunun için bir guard konur: "müzik durumundaysan bu 20. olay çalışsın". Benzer şekilde Beeping'e giden geçişte 20 olayı ve guard olarak `in Alarm` yazılmalıdır. Hoca bunun basit bir OCL tanımıyla da ifade edilebileceğini söyler.

Ardarda olaylar (broadcast cascade) da aktiviteleri koordine etmenin bir yoludur: bir iç olay yayınlanır ve bütün eş zamanlı makineler bütün olayları dinleyebildiği için kendi aralarında senkronize olabilir. Örnek: istasyon düğmesi çevrildiğinde sistemin üç tepki vermesi gerekir:

- Düğme fiziksel olarak yeni bir konuma gelmelidir; bu, düğmenin kendi makinesinde olur.
- Beyaz çubuk kaymalı, yeni bir frekans noktasını göstermelidir; bu ayrı bir makinededir ve ondan haberdar edilmelidir.
- Alıcıdaki radyo kanalı değişmelidir.

İstasyon makinesi bu olayları yayarak diğerlerinin tepki vermesini sağlar.

Her eş zamanlı makinenin varsayılan durumları da mutlaka belirlenmelidir. Kontrollü geçişler ("şu moddaysa şöyle geç") ve sayaç gibi iç olaylar yerleştirilmelidir. Böylece sistemin nasıl davranması gerektiği ve herhangi bir kaçak olmaması için nasıl bir model kurulacağı rahatça ifade edilir.

20.8 Genel diyagram

Genel diyagramda unplugged ve plugged durumları vardır; 5 ve 6 numaralı olaylarla (fişi tak, fişi çek) aralarında geçilir. Plugged durumunun içinde her şey bulunur: Display, Speaker ve diğerleri. Speaker'da Beep, Silent ve Playing durumları vardır; varsayılan sessizdir. Alarm olduğunda, alarm modundaysa 20 ile Beep'e geçilir. Ayrıca bir `e` olayıyla, alarm modundaysa Beep'e geçilir.

Bu `e` olayı snooze makinesinde yaratılır: normalde snooze durumunda değilizdir; snooze'a basılınca (21) snooze durumuna geçilir, 10 dakika sayılır, sonra normal durumuna dönülür ve `e` olayı yaratılır. `e` olayı aynı zamanda hoparlörün biplemesine yol açar.

Mekanizmalar, geçişlerin nedenleri ve varsayılanlar konduğunda tam bir durum modellemesi, yani davranış modellemesi ortaya konmuş olur.

> **Hocanın vurgusu:** Bu sayede sistemdeki nesnelerin ya da sınıfların belirli bir uyarana, belirli bir olaya karşı hangi tepkileri vermek zorunda olduğu, neleri dinlemek zorunda olduğu ve dinlediklerine göre başka neleri tetiklemek zorunda olduğu değişmez bir şekilde, kim okursa okusun aynı şekilde anlayacağı biçimde ifade edilmiş olur.

21. Geçerleme ve davranış modellemesinin adımları

Slayttaki geçerleme önerileri (ekrandan, 01:50:00):

- Durum diyagramı tamamlandıktan sonra geçerlemesi (validasyonu) yapılmalıdır.
- Bir ekip ile gözden geçirme yapılabilir: tüm kullanım senaryolarının üzerinden geçerek sistemin tam olarak beklenenleri yaptığı doğrulanır. Hoca, bunu tasarlayanlardan bağımsız bir ekibin yapmasının daha doğru olacağını ekler.
- Model kontrolü yapılabilir: diyagram gerçeklenip üzerinde testler koşturulur, olabilecek bütün durumları test eden senaryolarla olayların ve geçişlerin beklendiği şekilde oluşup oluşmadığı, eş zamanlı makinelerin düzgün çalışıp çalışmadığı test edilir.
- Bir simülatör yaratıp sistemin çalışması simüle edilebilir. Hoca, bir simülasyon ortamı yaratmanın oldukça maliyetli olacağını belirtir.
- Yeni sorular oluşursa yine kullanıcıya dönülür. Örneğin "alarm durumundayken snooze'a basıp öbür düğmeyi de çevirince ne olacak?" gibi henüz modellenmemiş karmaşık durumlar ortaya çıkabilir.

Hoca davranış modellemesini on adımda özetler:

1. Kullanım senaryoları yapılır ya da elde edilir.
2. Bu senaryolardan perceptler belirlenir.
3. Perceptlerle birlikte durumlar modellenir.
4. Harici kontroller ve uyaranlar (söylenmeyen, görülmeyen ya da olması beklenenler) belirlenir.
5. Durumlar, geçişler ve belirli davranışları tetiklemesi gereken sayaç gibi olaylar modellenir.
6. Uyaran-tepki ikilileri tablo hâline getirilir.
7. Zamanlayıcı gibi gerekli iç durumlar belirlenir.
8. Koordinasyon mekanizmaları (“şu olduğunda şu tepkiyi de yarat”) belirlenir.
9. Yapılması gereken eylem ve aktiviteler belirlenir.
10. Geçerleme yapılır.

22. Yazılım mimarisi nedir?

Hoca yeni slayt destesine (BLM5126-08) geçer: yazılım mimarisi deyince ne anlaşılmalı, temel kavramlar neler ve mimari stiller deyince hangi isimler görülür?

22.1 Üç bina benzetmesi

Slaytta üç yapı vardır: bir antik tapınak, Empire State’e benzeyen bir gökdelen ve bir şato. Üçünün de temel amacı aynıdır: barınma, ikamet sağlamak. Ama mimarileri çok farklıdır:

- **Antik tapınak:** Antik Yunan’da yapılmıştır. O zamanlar beton, çimento yoktu; taş ve mermer vardı. Sütunlar oyularak, taşlar üst üste konarak yapıldı. Yani eldeki araç, gereç ve bileşenler kullanıldı. Estetik kaygıları da vardır; hoca bunun göze çok daha güzel göründüğünü söyler.
- **Gökdelen:** farklı bir istek vardır. Taban alanı çok küçük, arsa çok değerlidir; arsaya verilen parayı çıkarmak için mümkün olduğunca çok kat çıkılmalıdır (arsa payı, inşaat payı, kâr payı). Burada işlevsel olmayan bir gereksinim vardır: belli bir alanda olabildiğince çok kişiyi barındırmak.
- **Şato:** amaç güvenliktir. İçeridekiler dışarıdan gelen her türlü saldırıya karşı korunur; duvarlar kale gibi sağlam, savunma için burçlar ve gözetleme kulesi vardır.

Gökdelen şatonun yerine konsaydı, çok fazla yüzey alanı olduğu için saldırganlar camları kolayca kırar ve güvenlik tehdit edilirdi; bu mimari o amaç için uygun değildir. Şato Beşinci Cadde’nin köşesine konamaz, çünkü orada minimum taban alanında maksimum yerleşim çözümü gerekir. Tapınak ise zamanının malzemeleriyle ancak bir-iki katlı olabilirdi.

22.2 Gayri resmî tanım

Yazılım mimarisi, bir tasarım probleminin en üst seviyedeki, soyut seviyedeki ifade tarzıdır. Pratikte kutular ve oklar kullanılarak sistemin temel bileşenleri ve bunların nasıl bağlı olacağı modellenir; oklar kontrol ya da veri akışları olabilir. Hoca, bir mimariyi ifade etmek için uluslararası, evrensel olarak kabul görmüş anlamsal bir uzlaşı olmadığını söyler: “ben böyle ifade ettim, mimarimi gösterdim” denebilir.

Gayri resmî tanım: bir sistemin bileşen alt sistemler ya da modüller hâlinde organize edilmesine, bölümlenmesine mimari tasarım denir. Genellikle katmanlar hâlinde, iterasyonlarla çoklu iyileştirmeler yapılarak oluşturulur. En soyut hâliyle sistemi oluşturan bileşenler, bunları oluşturan bileşenler ve alt sistemler, gerçekleştirilebilecek kadar düşük bir seviyeye kadar gösterilir. Genellikle önceden bilinen ve belli problemleri iyi çözdüğü söylenen mimari stiller kullanılır.

22.3 Analiz ve tasarım ilişkisi

Her şey analizle başlar. Analiz bileşenleri etkiler, çünkü sistemi oluşturacak parçalar analiz modeli işlenerek ortaya çıkarılır; müşterilerin değişimlere uyum sağlaması da buradan desteklenir. Ancak hoca şu uyarıları yapar:

- Bir analiz modeli, yani problem modeli ortaya konurken tasarım yaklaşımı ona dahil edilmemelidir. Analiz ve tasarım birbirinden mümkün olduğunca bağımsız tutulmalıdır: biri problemi ve alanı (domain) tanımlar, diğeri teknik tarafta bunun yazılım olarak nasıl çözüldüğünü gösterir.
- Belirli bir mimariye göre analiz yapılmamalıdır; problem gerçekten modellenmelidir.
- Analiz modeli donanım altyapılarına ve değişebilecek ortamlara karşı esnek olmalıdır. "Mainframe'de çalışacağım" varsayımıyla yola çıkılmamalı; model dağıtık bir sistemde ya da serverless bir mimaride de çalışabilmelidir.
- Sistemin gerektirdiği bütün bileşenlere analizde sahip olmak gerekmez. Örneğin koleksiyon sınıfları ya da işlem sınıfları tasarımda eklenir; analizde yalnız problemle ilgili şeyler tutulur.

23. Mimarinin resmî tanımları

23.1 USP'nin tanımı

Slayttaki tanım (ekrandan, 02:00:00):

> The set of significant **decisions** about the organization of a software system, the selection of the **structural elements** and their **interfaces** by which the system is composed, together with their behavior as specified in the **collaborations** among those elements, the **composition** of these structural and behavioral elements into progressively larger subsystems, and the architectural **style** that guides this organization: these elements and their interfaces, their collaborations, and their composition.
>
> Software architecture is concerned not only with structure and behavior but with **usage, functionality, performance, resilience, reuse, comprehensibility, economics and technology constraints and trade-offs, and aesthetic concerns**.

Hoca Türkçesini özetler: mimari kararlarla ilgilidir. Hangi bileşenler olacak, bunlar nasıl etkileşecek ve etkileşirken işlevsel olmayan gereksinimleri nasıl karşılayacak? Bileşenler yapısal elemanlardır ama arayüzleri de vardır: bulundukları dünyaya sundukları ve dünyadan bekledikleri. Hoca, sınıf diyagramlarının yanında fiş ve priz gibi çizilen çıkıntıları hatırlatır. Bileşenler işbirliği (collaboration) yaparak çözüm üretir; mimariyle bunlar giderek birleştirilip daha büyük alt sistemler ya da sistemler oluşturulur. Bu yapılırken önceden bilinen mimari stillerin rehberliğine dayanılır: tıpkı taş bina gibi, deneyimle ya da benzer sistemlerde iyi olduğu bilinen çözümler alınıp değiştirilerek ya da kullanılarak yeni sistemin mimarisi kurulur.

Yapısal ve davranışsal öğelerin yanında işlevsel olmayan gereksinimler de mimariyle mutlaka karşılanmalıdır. Hoca baştaki üç binayı hatırlatır: birinde minimum alanda maksimum insan, öbüründe güvenlik, diğerinde eldeki malzemeyle en etkin barınma çözümü hedeflenmişti.

> **Ek bilgi (derste anlatılmadı):** Slayttaki tanım büyük olasılıkla Booch, Rumbaugh ve Jacobson'ın UML ve Unified Software Development Process kitaplarında geçen tanımdır; "USP" kısaltması büyük olasılıkla Unified Software Process'i kasteder.

23.2 Diğer tanımlar

- **Perry ve Wolf:** tanıma gerekçe (rationale) eklerler. Mantıksal bir temel olmalıdır; kararlar belirli bir sebep-sonuç zinciri içinde mantıklı biçimde ifade edilmelidir.
- **IEEE:** sistemin bileşenlerinde, bunların birbirleriyle ve çevreyle ilişkilerinde somutlaşan organizasyonu ile tasarımını ve gelişimini yöneten ilkeler (organization of a system embodied in its components, their relationships to each other and to the environment, and the principles governing its design and development). Kavramlara nasıl karar verildiğinin ortaya konması da tanımda yer alır.
- **Değişimi öne alan tanım:** en zor değişen şeyleri saklamayın, merkezde tutun. Bir sistem beş yıl sonra büyük ihtimalle aynı kalmayacak, sürekli değişecektir; değişimler oldukça sistem kırılganlaşır ve beklenmedik davranışlar sergileyebilir.
- **Garlan ve Shaw:** en basit hâliyle mimari, bileşenler (components), bağlayıcılar (connectors) ve bunların düzenleşiminin (configuration) birleşkesidir.

Garlan ve Shaw'un üç unsuru:

- **Bileşenler:** işlemsel ya da veri elemanları ve arayüzleri.
- **Bağlayıcılar:** bileşenleri birbirine bağlar, birlikte çalışmalarını sağlar; aralarında bir protokolle haberleşmelerini sağlar. Kod da içerebilir; yalnız bir protokol tanımı değildir.
- **Düzenleşim (configuration):** bileşenlerin ve bağlayıcıların nasıl bir araya geleceğini, örneğin ikili bir bağlayıcıda hangi bileşenlerin bağlanacağını ya da topolojinin nasıl olacağını belirler.

24. Bileşenler, API'ler ve bağlayıcılar

24.1 Bileşen seçimi

Bileşenler UML'de zaten görülmüştü. Nasıl seçileceği konusunda çok kesin bir kural yoktur, ama genel olarak:

- Beklenen işlevselliği mutlaka karşılamalıdır.
- Mümkün olduğunca yeniden kullanılmalıdır; hazır olanlar ve kütüphaneler tercih edilmelidir.
- Paralelleştirme gibi fiziksel gereksinimler varsa bunlara uygun bileşenler seçilmelidir.
- En önemlisi ekibin deneyimidir. Hoca Conway yasasını anar: bir sistemin son hâli onu inşa eden organizasyonun yapısına bağlıdır. Ne kadar deneyimliyseniz o kadar iyi olur; deneyimsizseniz bileşenleri yanlış oluşturur ya da seçersiniz.
- Sistemin değişeceği yön (yörüngesi) önemlidir; çok dinamik, çok değişecek bir yapı varsa bileşen yapısı buna uygun kurulmalıdır.

  Ek bilgi (derste anlatılmadı): Conway yasası (Melvin Conway, 1968) büyük olasılıkla en çok "sistem tasarlayan organizasyonlar, kendi iletişim yapılarının kopyası olan tasarımlar üretir" biçiminde aktarılır. Hocanın ekip deneyimiyle ilişkilendirdiği nokta bu ilkenin bir yorumudur.

24.2 API

API (uygulama programlama arayüzü), bir bileşendeki ya da birimdeki elemanların nasıl çağrılıp kullanılacağına ilişkin tanımlardır. Genellikle bir programlama diliyle tanımlanır; buna language binding denir. OCL ya da ADL gibi özelleştirilmiş, bildirimsel notasyon dilleri de kullanılabilir. Özetle API, bileşenin sunduğu ve beklediği arayüzlerin tanımlandığı belge ya da birimdir. Bir mimariyi ifade ederken, özellikle bileşen yapılarını anlatmak için genellikle API'ler kullanılır.

24.3 Bağlayıcılar

Bağlayıcılar daha karmaşıktır; belirli kararlar alınmalı ve protokoller belirlenmelidir:

- Sıra nasıl olacak, sıra gözetilecek mi?
- İletim yönü nasıl olacak?
- Ortaya mı (yayın) konuşulacak, belirli bir hedefe mi?
- Bir şey yanlış gittiğinde hata nasıl düzeltilecek ya da geri alınacak?

En basit bağlayıcı prosedür çağrısıdır: bir taraf çağırır, öbürü geri dönüş değerini verir. İlişki asimetriktir; çağıran çağrıyı başlatır ve alt program bitene kadar başka bir işlem yapmadan bekler, yani senkrondur. Bunun gibi pek çok bağlayıcı olabilir. Düzenleşimin ayrıntısına hoca bu derste girmez.

25. Kavramsal mimari, sapma ve erozyon

Kavramsal mimari (conceptual architecture): belirsiz ya da yüksek seviyeli mimaridir. Sürecin en başında, gereksinimler bile tam anlaşılmadan üretilebilir: "şöyle bir bileşen yapısı, böyle bir bağlayıcı yapısı kurgulayabilirim" biçimindeki düşüncelerdir. Tamamen kavramsaldır ve değişebilir, ama bir kestirim yapmayı sağlar.

Planlanan ve inşa edilen mimari, yani hayaller ve gerçekler her zaman aynı olmaz. Bir uygulama gerçeklenirken baştaki kavramdan tamamen başka bir şey inşa edilebilir; örneğin hazır

bileşenler kullanılır ve sonradan planlanan sonuçtan uzaklaşıldığı fark edilir. Hazır bileşenleri kullanmak ne kadar kolay olsa da yapılması gereken işi tam yapamamakla sonuçlanabilir.

- **Mimari sapma (architectural drift):** bu uzaklaşma geliştirme sırasında oluşursa sapmadır.
- **Mimari erozyon (architectural erosion):** sistem kullanılmaya başladıktan sonra, bakım sürecinde oluşursa erozyondur. En kötüsü budur. Bakım ekibi günü kurtarmak için planlı olmayan değişiklikler yapabilir, örneğin bir bileşenin yerine bambaşka bir bileşen kullanabilir. Bunlar genellikle belgelenmez; bu yüzden sonraki bir değişimde yıkım çok daha hızlı olur. Bu nedenle buna sapma değil erozyon denir.

26. Mimari görünümler

Mimari görünüm (architectural view), mimariyi gösterme, diyagramlarla ifade etme şeklidir. Slayttaki maddeler (ekrandan, 02:10:00):

- Mimari tanım sadece bir diyagram değildir; bir kararlar kümesidir.
- Bu kümeyi tam olarak tanımlamak için pek çok diyagram ve belge üretilebilir; bunlara mimari görünümler denir.
- Bu küme geniş ve çok yönlü bir derleme içerebilir; pek çok mimari diyagram bir kısmını ifade eder.
- Katkı sağlayabilecek UML diyagramları: sınıf, bileşen, barındırma, nesne, paket, kullanım senaryosu, ardışıl, iletişim, etkileşim.

Hoca görünümlerle ilgili ayrı bir ders yapılacağını, bu derste bu konuya girmeyeceğini söyler.

27. Mimari stiller

Mimari stiller, daha önce “klişeler” ya da önceden bilinen iyi çözümler diye anılan şeylerdir. Tamamen isimlendirilmiş karar koleksiyonlarıdır: bileşenlerin neler olması gerektiğine ve etkileşimlerinin, bağlantılarının nasıl olması gerektiğine dair “şu şekilde bağlayın” türünden tavsiyelerden oluşur. Önceki deneyimlerden doğdukları ve belirli problemleri iyi çözdükleri bilindiği için, belirli bir problem için daha hızlı ve uygun bir mimari çözüm üretmeyi sağlarlar.

27.1 Hangi stil? Client-server

Hoca bir tanım verir ve hangi stile uyduğunu sorar (ekrandan, 02:11:50). Tanım ve cevap “Örnek Sorular” bölümündedir. Cevap client-server’dır.

Hoca client-server’ın temel tanımının bu olduğunu açıklar: servis veren ve servis alanlar fiziksel olarak ayrılır, ikisi bağımsız olarak ölçeklenir. Servis veren kime hizmet verdiğini bilmek zorunda değildir; kimlik belirtme süreci olabilir, onun dışında herkese şeffaf biçimde aynı hizmeti verir. İki taraf birbirinden bağımsız olarak değişebilir. Tek bir değişmez vardır: servis alan, servis aldığı yere bağımlıdır.

Hocanın benzetmesi bakkaldır: hep aynı bakkala gidersiniz, yalnız onun adresini bilirsiniz ve oradan hizmet alırsınız; ama o bakkal pek çok müşteriye hizmet verebilir. Bakkal taleplere yetişemezse yanına yeni bir bakkal açılması ya da büyük marketlerde kuyruk uzayınca yeni kasa açılması gibi, servis verenler dinamik olarak eklenir. Bir stili kullandım demek, ona uygun bileşen ve bağlayıcıları kullanıyorum demektir.

27.2 Stil kullanmanın faydası

Stil kullanınca deneyim kodlanmış olur: daha önce bilinenler ya da benzerleri yapılacağı belirtilir. Böylece genel geliştirme yükü azalır, gereksiz ya da alakasız yerlere gitmenin önüne geçilir. Örneğin sistem veri güdümlüyse (data-driven) bilinen bir MySQL çözümü doğrudan alınıp kullanılabilir. Mimari stiller, gerçekleştirme sırasında nasıl çalışılacağını da değiştirebilir.

28. Mimari stil kataloğu

Hoca uzun listeden en çok bilinen ya da en yaygın kullanılan stillerin isimlerini ve ana fikirlerini hızlıca tanıtır (ekrandan, 02:20:00 ve 02:25:00).

- **Abstract data type (soyut veri tipi):** temsilin gizlenmesi; verilere davranış eklenir ve bir paket hâlinde hizmet verirler. Çok ilkel bir sınıf tanımı gibidir; hoca bunu nesneye yönelimin belki ilk atası olarak görür ve günümüzde pek moda olmadığını söyler.
- **Batch sequential:** toplu ve sıralı işlemler için önerilmiş bir mimari. "Geçerle, düzenle, güncelle" (validate, edit, update) döngüsü işlemlerde sürekli tekrarlanır.
- **Blackboard:** sınıfta bir tahta vardır, herkes oraya istediğini ya da elde ettiği sonucu yazar. İşbirliği yapan ajanlar ve fırsatçı kontrolle çalışır: bir ajan beklediği sonucu tahtada görünce alır, işler ve başka bir sonucu yeniden bırakır; ya da bir istek bırakır, ajanlar dinler, "ben bunu yapabilirim" diyen yapıp sonucu ortama atar. Ortak bir veri havuzu gerekir.
- **Big ball of mud:** hocanın "en sevdiğim" dediği; bir stil değil, stil eksikliğinin tanımıdır. Ekip mimari tasarım çalışması yapmazsa büyük bir çamur topu, ne mimarisi ne düzgün bir çözümü olan, yalap şalap yapılmış bir uygulama çıkar.
- **Client-server:** yukarıda ayrıntısıyla görüldü; özellikle işlemlerde (transaction) çok kullanılır ve çok katmanlı olarak da modellenebilir.
- **Component-based:** yeniden kullanılabilir bileşenler ya da modüler yapıyla çözüm kurmayı önerir.
- **Coroutines:** simetrik etkileşim. Hoca `printf` örneğini verir: format bir string içinde tanımlanır, veri ayrıca verilir ve çıktı oluşturulurken veri o formata yerleştirilir. Bu şekilde simetrik olarak etkileşen ya da birlikte çalışması gereken yapılar varsa coroutine stili kullanılabilir.
- **Data-centric:** tamamen stored procedure kullanılmasını, işlemlerin veritabanında yapılmasını önerir; veri üzerindeki manipülasyon en hızlı şekilde yapılır. Hoca bunun da artık pek kullanılmadığını söyler.
- **Domain-driven:** belirli bir iş modeli için özel çözüm geliştirilecekse, örneğin askerî projelerde ya da kesin tanımlı vergi hesabı gibi uygulamalarda hâlâ kullanılır. Veri akışları iş modelini değiştirebiliyorsa bu stil kullanılabilir.
- **Implicit invocation (örtülü tetikleme):** bir kalıp (pattern) hâline de gelmiştir; publish-subscribe kalıbı denince hemen anlaşılır. Sistem olaylara verilen tepkiler hâlinde kurulur.
- **Layered (katmanlı):** soyut makineler kurulur; her katman kısıtlı görünürlükle yalnız bir üstündekine hizmet eder. Hoca bunun "n-tier" gibi düşünülmemesini, onun daha çok client-server mimarisi olduğunu söyler. Daha çok işletim sistemi gibidir: katman katman, X11 grafik arayüzü gibi her katman üstündekine hizmet eder; parçalar pasta gibi dizilir.
- **Master control:** hocanın bildiği kadarıyla yaratılan ilk stildir ve çok eskidir. Hiyerarşik olarak alttaki yordamlar çağrılır ve geri dönülür; mainframe programları gibi.
- **Message bus:** asenkron mesajlar ortak bir veri yolundan geçer. Mikroservis mimarisindeki mesaj kuyrukları buradan yola çıkılarak oluşturulmuştur.
- **Mobile code:** özellikle mobil cihazlar için istek üzerine kod, uzaktan değerlendirme ve mobil ajan. Uzaktaki bir mobil ajanın merkezdeki ya da uzaktaki bir kodu ya da prosedürü çalıştırarak tetiklemesi; ajan tabanlı bir mimari.
- **Object oriented:** nesneye yönelik programlamadan biraz farklıdır. Asenkron mesaj iletimi ve bağımsız kontrol iş parçacığı vardır. Nesneler vardır ama hepsinin varlığı bağımsızdır, kontrolleri kendilerindedir; birbirlerine asenkron mesaj yollayarak, işbirliği içinde problemi çözerler. Hoca bunu kavramları nesneler hâlinde modelleyip bir işbirliği diyagramında birbirine çağrılar attırarak bir senaryodaki sorumluluğu yerine getirmeye benzetir.
- **Peer-to-peer:** eşit ortaklar bir servisin sorumluluğunu paylaşır; yönetici olmadan herkesin eşit seviyede olduğu bir işlem dağıtımı. Hoca eDonkey, eMule ve torrentleri örnek verir.
- **Plug-in:** kayıtlı üçüncü parti eklentilerle sistemin büyütülmesi; Eclipse tarzı ortamlar. SAP de modül eklenerek büyüyen bu şekilde modellenmiştir. IDE'lerde de çok kullanılır.
- **Pipe and filter:** Python'da veri işleme yapılırken yapılan şeydir. Önce veri okunur, ön işlemede temizlenir, çıktısı PCA gibi bir boyut dönüşümüne verilir, onun çıktısı bir sınıflama ya da kümeleme algoritmasına, onunki bir değerlendiriciye (train-test) verilir ve performans alınır; sonunda hata eğrileri gibi grafikler çizilir. Arada filtreler, filtrelerin arasında borular vardır; veri bir yerden girer ve tek yönde filtrelerden geçerek sona akar.

- **Process control:** donanımı kontrol eden yazılımlarda kullanılır; bir geri besleme döngüsü tanımlar. Arabalardaki hız sabitleyici gibi: 70'e sabitlenince 70'e kadar gaz verir, 70 olunca gazdan çeker; hız düşünce yine gaz verir, çok artarsa fren yapar.
- **Production systems:** kural tabanlı ya da koşullu çalışma; yapay zekâdan alınmıştır. Kontrol akışı hakkında tam bir fikir yoksa kullanılabilir. Hoca kendisinin pek kullanmadığını söyler.
- **REST (Representational State Transfer):** günümüzde çok modadır; HTTP uygulamalarının çoğu bu mimaride gerçekleştirilir. Yine bir client-server ilişkisi vardır ama katmanlı ve durumsuzdur: aynı istemciden gelse bile her istek öncekilerle ilişkisi olmadan bağımsız ele alınır. Önbelleklenebilir, dağıtık hiper ortamlar kurulmasını sağlar.
- **Service-oriented (SOA):** sistemin işlevleri ayrık servislerdedir; sistem bir servis kümesidir. Servisler gevşek ilişkili, durumsuz, keşfedilebilir ve kontrat tanımlıdır. Bunun için WSDL gibi diller tanımlanmıştır: bir servisle nasıl haberleşileceği bir yerden okunup anlaşılabilir, bir servisin nerede, kim olduğu sorgulanıp bulunabilir.
- **Shared nothing (sharded):** düğümler arası paylaşım olmayan dağıtık veritabanı.
- **State-transition systems:** reaktif, asenkron olaylara tepki veren sistemler; örneğin GUI'ler ve kullanıcının yarattığı olaylar. Gerçek zamanlı da olabilir. Hoca, derste yapılan durum geçiş modellemesinin tam olarak bunun tanımı olduğunu söyler.
- **Shared memory:** senkronizasyon için kilit mekanizması; tüm işlevler ortak bir bilgi havuzunu kullanır. Herkes ortak veriye eriştiği için güvenlik gerekiyorsa kullanılmamalıdır. Blackboard'a yakındır.
- **Table-driven interpreter:** çözümle ve dağıt. İstekler belirli bir dille kodlanır, bir yorumlayıcı bunları çözümleyip ilgili yordamlara gönderir; istek kuyruğunu işleyerek işlerin yapılmasını sağlar.

Hoca kataloğun bundan ibaret olmadığını, ama en çok bilinen ve tarih içinde karşımıza çıkan yaklaşımların bunlar olduğunu söyler. Günümüzde farklı mimarilere geçiş vardır. Listede sayılmayan ama pek çok tarafıyla bunların bazılarından miras alan mikroservisler, service-oriented'dan sonra gelir. Bulutun gelişmesiyle serverless mimariler ortaya çıkmıştır; bunlar şu anda en modern mimari tasarımlar olarak kullanılır.

29. Heterojen sistemler, ADL'ler ve mimari değerlendirme

29.1 Heterojen sistemler

Bir stili kullanmak her zaman bir problemi doğrudan çözmez; pek çok sorun kalmaya devam eder. Bu yüzden problemler genellikle birden fazla stil birlikte kullanılarak çözülür; bunlara **heterojen sistemler** denir.

Hoca bir askerî uçak örneği verir: F-16'nın bomba atan, radar taşıyan, fotoğraf çeken, radar sinyali bozan ya da eğitim için olan versiyonları vardır. Ama uçak aynı uçaktır; yüzde 90-95'i ve kontrol mekanizması değişmez. Farktan çok benzerlik içeren, az farklılıklı çeşitli versiyonları olan bu tür sistemlerde ortak kısım bir mimariyle tanımlanır, özelleşen kısım varsa ek bir mimariyle tanımlanır. Mimaride ilerledikçe stillerin tanımlarının daha kesin olması da gerekir.

29.2 Mimari tanımlama dilleri (ADL)

Mimariyi tanımlamak için bazen bir notasyona, bir dile ihtiyaç duyulur; buna ADL (Architecture Description Language) denir. ADL'ler araç desteği de sağlar: örneğin çizilen bir diyagramın bir stile uygun olup olmadığı daha otomatik kontrol edilebilir, simülasyonlar bu araçlarla yapılabilir. Slaytta pek çok ADL örneği vardır; hoca zaman kalırsa bunlardan birine ileride bakılacağını söyler.

29.3 Mimari değerlendirme

Bir mimari uygulandıktan sonra doğru mimarinin uygulanıp uygulanmadığı kontrol edilebilir:

- **Architectural Review Board:** daha gayri resmî bir yöntem.
- **SAAM:** hoca "Software Architecture Assessment Method" diye açar.
- **ATAM (Architecture Tradeoff Analysis Method):** daha resmî bir yöntem; Carnegie Mellon'un ürettiği.

Hoca bunlara çok fazla girmez; yalnız doğru stilin uygulanıp uygulanmadığını geçerlemeyle kontrol eden mekanizmaların tanımlı olduğunu bilmek için söyler.

> **Ders sonrası not:** SAAM'ın yaygın açılımı büyük olasılıkla "Software Architecture Analysis Method"dur. SAAM ve ATAM büyük olasılıkla Carnegie Mellon Üniversitesi'ndeki Yazılım Mühendisliği Enstitüsü (SEI) tarafından geliştirilmiştir.

29.4 Özet

Slayttaki özet (ekrandan, 02:30:00):

- Yazılım mimarisine ve mimari stillere bir giriş yapıldı; ilerleyen derslerde daha ayrıntılı bakılacak.
- Nihai hedef yüksek kaliteli sistemler üretmek ve maliyetleri azaltmaktır.
- Bunu yapmanın en iyi yolu problemlerin erkenden belirlenmesidir. Bunun için sistemin neye benzeyeceği en baştan belirlenmeli; sorunlar ve mantıklı çözümleri ortaya konmalıdır.
- Elimizdeki hazır çözümler ve araçlar çözümde kullanılabilmelidir.
- Tüm bunlarla, tüm paydaşlara hızlı ve etkin şekilde fikirlerimizi aktarabileceğimiz kadar soyut bir mimari inşa etmek isteriz.

Örnek Sorular

Soru 1: Durum mu olay mı?

Kaynak: (ekrandan, 00:20:00)

Aşağıdakilerin her biri bir durum mu, bir olay mı, yoksa hiçbiri mi?

- Jesse Owens'ın Berlin'de 1936'da 100 metre olimpiyat şampiyonu olması
- Mor renk
- Binanın yangın söndürme sisteminin bir yangın nedeniyle devreye girmesi
- Telefonun çalması
- Yağmurlu bir gün
- Saatin zilinin 12'yi vurması
- Uluslararası Uzay İstasyonu
- Bilgisayarınızın ekran koruyucusu açılıyor

Çözüm (derste verildi): Owens'ın şampiyon olması durumdur (rekor kırması olaydır). Mor renk hiçbiridir, yalnız bir özelliğin değeridir. Yangın söndürme sisteminin devreye girmesi olaydır. Telefonun çalması olaydır. Yağmurlu bir gün durumdur. Saatin 12'yi vurması olaydır. Uluslararası Uzay İstasyonu hiçbiridir, yalnız çok karmaşık bir nesnedir. Ekran koruyucusunun açılması için slaytta cevap yoktur; hoca bunu bilgisayarı bekleme durumuna geçiren tetikleyici bir olay olarak açıklar.

Soru 2: Kasabadaki arabaların durum sayısı

Kaynak: (derste sözlü, 00:10:57)

10 sokak ve 8 caddeden oluşan ızgara biçimli bir kasabada bir araba herhangi bir kesişimde bulunabilir. Tek araba ve iki araba için kaç durum vardır?

Çözüm (derste verildi): Tek araba için $10 \times 8 = 80$ durum. İki araba için kabaca $80 \times 80 = 6400$ durum.

> **Çözüm (derste verilmedi):** İki araba aynı kesişimde bulunamıyorsa çıkarılması gereken hâl sayısı 80'dir (hocanın dediği gibi 1 değil). Arabalar ayırt edilebilirse $80 \times 79 = 6320$, ayırt edilemezse $\binom{80}{2} = 3160$ durum vardır (python ile doğrulandı).

Soru 3: SOS tahtasının durum sayısı

Kaynak: (derste sözlü, 00:13:11)

3x3'lük bir SOS tahtasında her göz S, O ya da boş olabilir. Oyun kuralları düşünülmezse kaç olası durum vardır?

Çözüm (derste verildi): $3^9 = 19683$ durum (python ile doğrulandı).

Soru 4: Garaj kapısında düğmeye basılınca ne olmalı?

Kaynak: (derste sözlü, 00:25:45)

Tek düğmeli bir garaj kapısında kapı ve motor durmuşken, kapı yukarı çıkarken ve kapı aşağı inerken düğmeye basılırsa ne olmalıdır?

Çözüm (derste verildi): Kapı dururken düğmeye basılırsa ters yönde hareket etmelidir. Yukarı çıkarken basılırsa durmalıdır. Aşağı inerken basılırsa durmalıdır ("altından kedi geçiyor"). Aynı olaya mevcut duruma göre farklı tepki verildiği için bu ardışık bir sistemdir; ayrıntılı geçişler 8. bölümdeki tablodadır.

Soru 5: Zaman göstergesinde kaç farklı durum olabilir?

Kaynak: (ekrandan, 01:30:00)

Radyo saatin ekranında iki ışık düşünülmezse, saatler 01..12 ve dakikalar 00..59 arasındaysa kaç farklı durum vardır? Durum diyagramında nasıl modellenir?

Çözüm (derste verildi): $12 \times 60 = 720$ durum; 720 durumlu bir sonlu durum makinesiyle modellenebilir. Durum diyagramında iki eş zamanlı makineyle $12 + 60 = 72$ durumda anlatılabilir. Bu bile fazla olduğu için "saat" tek bir düğüme soyutlanır ve 720 durumun beklendiği şekilde oluştuğu varsayılır (python ile doğrulandı).

Soru 6: Hangi mimari stil?

Kaynak: (ekrandan, 02:11:50)

- Servis veren ve servis alan yazılım bileşenleri fiziksel olarak ayrıdır (ayrı makinelerde). Bu sayede iki taraf da bağımsız olarak ölçeklendirilebilir.
- Servis veren bileşenler, alanlar kimliklerini belirtmediği sürece servis alanların kim olduğunu bilmez. Bu sayede şeffaf olarak değişebilecek istem yapanlara hizmet verebilirler.
- Servis alanlar birbirinden izoledir; bu sayede bağımsız olarak ekleme, çıkarma, değişim yaşayabilirler. Servis alanlar sadece servis verenlere bağımlıdır.
- Servise talep belirli bir değerin üzerine çıktığında servis verenler dinamik olarak eklenebilir ve mevcut hizmet verenlerin yükünü azaltabilir.

Hangisi? Blackboard, Client Server, Implicit Invocation, Layered, Object Oriented, Pipe and Filter.

Çözüm (derste verildi): Client-server.

Dersin Toparlaması

Dersin ilk bölümünde sistemin dış uyarılara nasıl tepki vereceğini modellemek için durum ve olay kavramları kuruldu. Durum uzayının üstel büyümesi (durum patlaması) yüzünden her durumu tek tek yazmak mümkün değildir. Kombinatoryal sistemler karar tablosu ya da ağacıyla, ardışık sistemler sonlu durum makinesiyle modellenir. Eş zamanlı sistemler için Harel'in durum diyagramları kullanılır: iç içe gruplama, eş zamanlı parçalar, koşullu geçişler, entry/exit/do/defer, olay parametreleri, yayım olayları ve history ile karmaşıklık yönetilir. Casio saat ve radyo saat örnekleri, perceptlerden başlayıp olaylar, iç sayaçlar, koordinasyon ve geçerlemeyle biten on adımlı davranış modellemesi sürecini gösterdi.

İkinci bölümde yazılım mimarisinin gayri resmî ve resmî tanımları, Garlan ve Shaw'un bileşen-bağlayıcı-düzenleşim üçlüsü, kavramsal mimari, mimari sapma ve erozyon, mimari görünümler ve mimari stil kataloğu tanıtıldı. Mimari görünümler ayrı bir derste işlenecek. Hoca, sonraki derslerden itibaren mikroservis mimarisinin sunumlarla daha ayrıntılı inceleneceğini söyler.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.